

Export SAP Tutorial to PDF

<https://gemini.google.com/app/9019699c342d67c8>

User prompt: Please help me scan the content in this page and export it to PDF file-- <https://developers.sap.com/tutorials/abap-environment-sbpa-workflow-trigger.html>

Response: Your PDF document has been created and exported successfully.

[file-tag: code-generated-file-a51a3db4-32e0-4836-bbd0-19c362d15a0a]

Summary of Document Content: **Create a Process Trigger**

Source URL: <https://developers.sap.com/tutorials/abap-environment-sbpa-workflow-trigger.html>

Metadata: Beginner | 15 Min | SAP Build Process Automation & ABAP Environment

Key Highlights & Steps Included in PDF:

1. Overview of SBPA Process Triggers:

- **Form Triggers:** Start a process via an interactive request form.
- **API Triggers:** Start a process programmatically via external API calls (e.g. from BTP ABAP Environment or S/4HANA).
- **Event Triggers:** Listen to backend business events and trigger processes automatically.

2. Step 1: Create Data Type (Travel Context):

- Fields defined: Travel id, Agency Name, Booking fee, Currency Code, Customer Name, Total price (all String type).
- Guidance on precise field casing and spelling required for the API payload.

3. Step 2: Create API Trigger (Travel Approval Trigger):

- Configuring the process Travel Approval Processing with a **Call an API** trigger type.

4. Step 3: Configure Process Inputs:

- Mapping the Travel Context data type to process input variables.
- Notes on trigger lifecycle (deactivate vs. delete) and release/deployment requirements.

5. Self-Assessment / Quiz:

- Overview of trigger types available for SAP Build Process Automation.

User prompt: continue for this page: <https://developers.sap.com/tutorials/abap-environment-sbpa-workflow-form.html>

Response: I have scanned the SAP tutorial page (**Create and Configure Approval Form**) and exported the content into a formatted PDF document.

[file-tag: code-generated-file-ca58f617-64db-4cc2-bbb2-358886e2352f]

Summary of Document Content: **Create and Configure Approval Form**

Source URL: <https://developers.sap.com/tutorials/abap-environment-sbpa-workflow-form.html>

Metadata: Beginner | 15 Min | SAP Build Process Automation & BTP ABAP Environment

Key Highlights & Steps Included in PDF:

1. Overview:

- Building interactive forms routed to users in **My Inbox** for decision-making (approvals/rejections) without requiring coding.

2. Step 1: Create and Design the Approval Form:

- Creating a blank approval form named Approval Form.
- **Form Structure:** Adding Headlines (Travel Approval) and Paragraph descriptions.
- **Read-Only Context Fields:** Mapping input fields (Travel ID, Agency Name, Booking Fees, Currency Code, Customer Name, Total Price) marked as **Read Only**.
- **Interactive User Fields:** Adding a required Comment input field for approvers.

3. Step 2: Configure the Approval Form in Process Flow:

- **Subject & Description:** Combining plain text and dynamic Process Content attributes (e.g., Review and approve travel Travel id [Travel ID] for customer [Customer Name]).
- **Recipients:** Binding the recipient to Process Started By or specifying user email addresses.
- **Due Date:** Setting a Static Duration of 2 days.
- **Inputs Mapping:** Binding all form fields to their corresponding Process Content entries.

4. Self-Assessment / Quiz:

- **Q:** Can you combine plain text and process content attributes in the subject of a form?
- **A: Yes, of course** (e.g., mixing static text with dynamic process attributes).

User prompt: For this page: <https://developers.sap.com/tutorials/abap-environment-sbpa-workflow-action.html>

Response: Your PDF document has been created and exported successfully.

[file-tag: code-generated-file-d5ba3d6e-1f67-4b8a-ba9b-39a88e542c0a]

Summary of Document Content: Create and Configure Action in the Process Flow

Source URL: <https://developers.sap.com/tutorials/abap-environment-sbpa-workflow-action.html>

Metadata: Beginner | 30 Min | SAP Build Process Automation & BTP ABAP Environment

Key Highlights & Steps Included in PDF:

1. Overview:

- **Actions** allow you to connect your workflow processes to external systems (e.g., SAP S/4HANA, SAP BTP ABAP Platform, Ariba) for API operations like GET, POST, or PATCH.

2. Step 1: Create an Action Project:

- Upload an Open API specification in the SAP Build Lobby (under **Connectors > Actions**).
- Configure the Action project (Name: Workflow Notification) and add the necessary service operation (e.g., Post).

3. Step 2: Add Action to the Business Process:

- Access your business project (e.g., RAP110_Travel_Workflow) and open the process builder.
- Attach the Action to both the **Approve** and **Reject** branches of the Approval form step to trigger callbacks upon workflow completion.

4. Step 3: Configure the Action:

- **Destination:** Create a Callback_Destination variable to enable connectivity to the external system.
- **Inputs Mapping:** Bind Action parameters (like WorkflowInstanceId) to process metadata (e.g., **Process Instance ID**).
- **Branch Logic:** For the "Reject" branch, configure the Action to pass the rejection comment (mapped from the form's comment field) to the external system as the Outcome.

User prompt: For this page : <https://developers.sap.com/tutorials/abap-environment-sbpa-workflow-release-deploy.html>

Response: An offline reference document and PDF summary have been compiled for the tutorial:

Tutorial: [Release and Deploy Business Process Project](#)

Metadata: Beginner | 10 min | SAP Build Process Automation & SAP BTP ABAP Environment

Key Concepts & Step-by-Step Breakdown

1. Core Workflow Concepts

- **Release:** Creates a versioned snapshot of the project. Uses semantic versioning (X.Y.Z):
 - **Bug Fix (Z):** Patch updates and bug fixes.
 - **Minor Changes (Y):** Incremental feature updates.

- **Major Changes (X):** Breaking architectural/workflow changes.
- **Deploy:** Instantiates the released process into the target environment (e.g., Public) and assigns target runtime variables (such as BTP destinations).

2. Step 1: Release the Business Process Project

1. Open the process project (RAP110_Travel_Workflow) in Process Builder.
2. Click **Release** in the top right corner.
3. Enter optional **Release Notes / Comments**.
4. Click **Release**. Initial release versions start at 1.0.0.

3. Step 2: Deploy the Released Project

1. From the released project view, click **Deploy** > select environment **Public** > click **Deploy**.
2. **Configure Runtime Variables:**
 - In the destination drop-down, select the destination (e.g., abap-cloud-dev-steampunk-canary) for the Callback_Destination variable created in SAP BTP Subaccount.
3. Click **Deploy**.
4. Upon successful deployment, the project is live and ready for execution and monitoring.

PDF & Document Download

[file-tag: code-generated-file-83fb018b-9280-468c-97da-08715a305b93]

User prompt: For this page: <https://developers.sap.com/tutorials/abap-environment-sbpa-workflow-handler-class.html>

Response: An offline PDF document has been created for the tutorial:

Tutorial: [Create Handler Class to Process Callback from Workflow Process](#)

Metadata: Beginner | 20 min | SAP BTP ABAP Environment & SBPA Integration

Key Highlights & Step-by-Step Summary

1. Overview & Objective

When an SBPA workflow decision (Approve / Reject) is submitted by an end user, the process flow executes an Action callback. This ABAP Handler class handles incoming HTTP REST callback requests and updates the RAP Travel Business Object status in SAP BTP ABAP Environment using EML.

2. Step 1: Create the ABAP Class in ADT

1. Open ABAP Development Tools (ADT) and navigate to your development package (e.g., ZRAP110_###).
2. Create a new ABAP Class:
 - **Name:** ZCL_RAP110_WORKFLOW_HANDLER_###
 - **Description:** Workflow Callback Handler Class

3. Step 2: Implement HTTP Service Extension

1. Implement interface IF_HTTP_SERVICE_EXTENSION in the public section of the class.
2. In method if_http_service_extension~handle_request:
 - Read the incoming JSON body containing WorkflowInstanceId and Outcome / Comment.
 - Parse parameters to determine whether the request was approved or rejected.

4. Step 3: Update Business Object via EML

1. Execute **EML (Entity Manipulation Language)** statement MODIFY ENTITIES OF ZRAP110_I_TRAVEL_### to update overall travel status (Accepted / Rejected).
2. Perform COMMIT ENTITIES to save changes to the database.

3. Return HTTP 200 OK status in the response object.

PDF & Document Download

[file-tag: code-generated-file-9c68cccf-fd1f-456b-b99e-0d07e81bfd99]

User prompt: For this page: <https://developers.sap.com/tutorials/abap-environment-sbpa-workflow-extend-rap-app.html>

Response: An offline PDF reference document has been compiled for the tutorial:

Tutorial: [Integrate SBPA Workflow into RAP Business Object Application](#)

Metadata: Beginner | 25 min | SAP BTP ABAP Environment & SBPA Integration

Key Summary & Implementation Steps

1. Overview & Objective

This tutorial completes the end-to-end integration by adding a custom action to the RESTful Application Programming (RAP) Travel Business Object (BO). Executing this action triggers the deployed SAP Build Process Automation (SBPA) workflow via REST API and sets the local travel status to "In Approval".

2. Step-by-Step Integration

- **Step 1: Declare Action in Behavior Definition** Add a custom instance action (`createApprovalWorkflow`) in `ZRAP110_I_TRAVEL_###`.
- **Step 2: Implement SBPA Trigger in Behavior Implementation Class** Inside `BP_ZRAP110_I_TRAVEL_###`:
 1. Read instance data via Entity Manipulation Language (EML).
 2. Build JSON payload containing the SBPA process definition ID and input parameters.
 3. Call the SBPA trigger API via HTTP client / Destination Service.
 4. Update travel status to "In Approval" (0) via `MODIFY ENTITIES`.
- **Step 3: Expose Action & Add UI Button**
 - Expose use action `createApprovalWorkflow`; in Projection Behavior Definition (`ZRAP110_C_TRAVEL_###`).
 - Add `@UI.lineItem` / `@UI.identification` annotations with type: `#FOR_ACTION` in CDS Metadata Extension to render the **Send for Approval** button.
- **Step 4: End-to-End Testing**
 1. Click **Send for Approval** in the Fiori Elements Travel Preview.
 2. Confirm new process instance in **SBPA Monitoring**.
 3. Approve or Reject the task form in **My Inbox**.
 4. Verify the RAP BO status is updated to *Accepted* (A) or *Rejected* (X) via the HTTP callback handler class.

PDF & Document Download

[file-tag: code-generated-file-3061354c-b95b-4a37-999d-dd4f132e1d78]

User prompt: For this page : <https://developers.sap.com/tutorials/abap-environment-sbpa-workflow-run-monitor-troubleshoot.html>

Response: An offline PDF reference document has been generated for the tutorial:

Tutorial: [Run, Monitor, and Troubleshoot SBPA Workflow Integration](#)

Metadata: Beginner | 15 min | SAP BTP ABAP Environment & SBPA Integration

Key Summary & Practical Guide

1. Execution & Process Trigger

- Launch the Fiori Elements Travel Preview from ADT.
- Select an *Open* Travel instance and click **Send for Approval**.

- Confirm that the RAP Travel BO status changes to **In Approval** (0).

2. Monitoring & Approval Processing

- Navigate to **SAP Build Process Automation Lobby > Monitoring**.
- Under **Process and Workflow Instances**, track the active execution flow diagram.
- Open **My Inbox** in SAP BTP to process, approve, or reject the pending approval task form.

3. Verification & Callback Validation

- Refresh the Fiori Elements Travel application.
- Verify that the callback handler class (ZCL_RAP110_WORKFLOW_HANDLER_###) updated the Travel BO status to **Accepted** (A) or **Rejected** (X).

4. Troubleshooting Steps

- **Workflow Errors:** Filter for *Erroneous* instances in SBPA Monitoring to inspect failed step logs.
- **Connection / Auth Failures:** Verify BTP Destinations and OAuth/Service Keys between ABAP System and SBPA.
- **Callback Execution Issues:** Check ADT Gateway traces/logs if the HTTP callback handler fails during EML database update.

PDF & Document Download

[file-tag: code-generated-file-353d8cc5-71f4-4703-8de7-deb6da8104e2]

User prompt: please help provide clear logic for the method which will consume to call api trigger sap SBPA workflow process from sap s4hana abap

Response: To trigger an SAP Build Process Automation (SBPA) workflow from SAP S/4HANA ABAP, your ABAP logic follows a 4-step sequence:

Authenticate (Fetch OAuth Token) → Build Payload → Call API Endpoint → Parse Response.

Below is the architectural workflow, prerequisites, and code logic for implementing this cleanly in ABAP.

Prerequisites & Endpoints

You need the details from your **SBPA Service Key** generated in SAP BTP:

- **OAuth Token URL:** `https://<subdomain>.authentication.<region>[.hana.ondemand.com/oauth/token]`
(`https://.hana.ondemand.com/oauth/token`)
- **Workflow REST API URL:** `https://spa-api-gateway-b2b-.../workflow/rest/v1/workflow-instances` (or public endpoint `/public/workflow/v1/workflow-instances`)
- **Client Credentials:** `clientid` and `clientsecret`

Method Signature & Data Flow

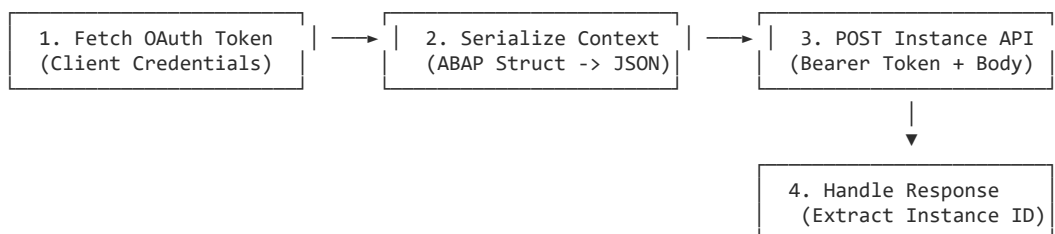
Method Inputs:

- `iv_definition_id` (TYPE string): The Process Definition ID from SBPA (e.g., `us10.mytenant.salesorderapproval.process`).
- `is_context` (TYPE any / ABAP Structure): Data structure to send into the workflow context.

Method Outputs:

- `ev_instance_id` (TYPE string): Workflow Instance ID returned by BTP.
- `ev_status_code` (TYPE i): HTTP return status (e.g., 201 for created).

Logic Execution Steps



Step 1: Fetch OAuth 2.0 Access Token

Perform an HTTP POST request to the BTP UAA token endpoint using `grant_type=client_credentials`. (Note: If configured via SM59 destination with OAuth2, SAP handles token generation automatically, allowing you to skip manual token fetching.)

Step 2: Build JSON Payload

Construct the mandatory JSON structure expected by SBPA:

```
{
  "definitionId": "us10.mytenant.salesorderapproval.process",
  "context": {
    "SalesOrder": "10000123",
    "Amount": 15000,
    "RequestedBy": "USER_A"
  }
}
```

Step 3: Trigger Workflow Instance

Execute an HTTP POST to the SBPA workflow instances endpoint passing `Authorization: Bearer <access_token>` in the header and the payload in the body.

Step 4: Process Response

A successful workflow trigger returns HTTP Status 201 Created with a JSON payload containing `id` and `status` (RUNNING).

Implementation Code (Modern ABAP / S/4HANA)

Here is the modular ABAP logic using standard HTTP client classes:

```
CLASS zcl_sbpa_workflow_utility DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    TYPES:
      BEGIN OF ty_workflow_response,
        id TYPE string,
        definition_id TYPE string,
        status TYPE string,
      END OF ty_workflow_response .

  CLASS-METHODS trigger_process
    IMPORTING
      !iv_definition_id TYPE string
      !is_context TYPE any
    EXPORTING
      !ev_instance_id TYPE string
    RAISING
      cx_web_http_client_error
      cx_sxml_parse_error .
ENDCLASS.

CLASS zcl_sbpa_workflow_utility IMPLEMENTATION.

  METHOD trigger_process.

    " -----
    " 1. Constants (Replace with Config/Table values or SM59 destination)
    " -----
    DATA(lv_token_url) = 'https://<your-subdomain>.authentication.<region>.hana.ondemand.com/oauth/token'.
    DATA(lv_api_url) = 'https://spa-api-gateway-b2b-.../workflow/rest/v1/workflow-instances'.
    DATA(lv_client_id) = 'YOUR_BTP_CLIENT_ID'.
    DATA(lv_client_sec) = 'YOUR_BTP_CLIENT_SECRET'.

    " -----
    " 2. Fetch OAuth Token via HTTP Client
    " -----
    DATA(lo_auth_client) = cl_web_http_client_manager=>create_by_http_destination(
      cl_http_destination_provider=>create_by_url( lv_token_url ) ).

    DATA(lo_auth_req) = lo_auth_client->get_http_request( ).
```

```

lo_auth_req->set_authorization_basic( username = lv_client_id password = lv_client_sec ).
lo_auth_req->set_form_field( i_name = 'grant_type' i_value = 'client_credentials' ).

DATA(lo_auth_res) = lo_auth_client->execute( if_web_http_client=>post ).
DATA(lv_auth_json) = lo_auth_res->get_text( ).

" Extract access_token from response
DATA: BEGIN OF ls_token_resp,
      access_token TYPE string,
    END OF ls_token_resp.

/ui2/cl_json=>deserialize( EXPORTING json = lv_auth_json CHANGING data = ls_token_resp ).

IF ls_token_resp-access_token IS INITIAL.
  " Raise exception / handle authentication failure
  RETURN.
ENDIF.

" -----
" 3. Prepare SBPA Payload
" -----
" Convert ABAP context structure to JSON string
DATA(lv_context_json) = /ui2/cl_json=>serialize( data = is_context compress = abap_true pretty_name = /ui2/cl_json=>

" Construct full API request body
DATA(lv_payload_json) = |\{ "definitionId": "{ iv_definition_id }", "context": { lv_context_json } \|.

" -----
" 4. Call SBPA Workflow Trigger API
" -----
DATA(lo_api_client) = cl_web_http_client_manager=>create_by_http_destination(
                        cl_http_destination_provider=>create_by_url( lv_api_url ) ).

DATA(lo_api_req) = lo_api_client->get_http_request( ).
lo_api_req->set_header_field( i_name = 'Authorization' i_value = |Bearer { ls_token_resp-access_token }| ).
lo_api_req->set_header_field( i_name = 'Content-Type' i_value = 'application/json' ).
lo_api_req->set_text( lv_payload_json ).

DATA(lo_api_res) = lo_api_client->execute( if_web_http_client=>post ).
DATA(lv_status_code) = lo_api_res->get_status( )-code.
DATA(lv_res_json)    = lo_api_res->get_text( ).

" -----
" 5. Parse Response Instance ID
" -----
IF lv_status_code = 201.
  DATA ls_workflow_res TYPE ty_workflow_response.
  /ui2/cl_json=>deserialize( EXPORTING json = lv_res_json CHANGING data = ls_workflow_res ).
  ev_instance_id = ls_workflow_res-id.
ELSE.
  " Handle error / log HTTP response
ENDIF.

lo_api_client->close( ).
lo_auth_client->close( ).

ENDMETHOD.
ENDCLASS.

```

Best Practices

1. **Use SM59 HTTP Destinations:** Avoid hardcoding URLs and client credentials in ABAP code. Maintain a destination in transaction SM59 with **OAuth2 Client** configured. This eliminates manual token parsing and automatically handles token refresh logic.
2. **Asynchronous Execution:** If triggering this from a high-frequency business transaction (e.g., SAVE BAdI or User Exit), call this method inside a **Background Task** (IN BACKGROUND TASK) or via **Application Job / qRFC** to prevent network latency from blocking the main user session.
3. **Application Logging (SLG1):** Wrap the API response in BAL_LOG_CREATE / BAL_LOG_MSG_ADD calls to log SBPA instance_id or HTTP error details for auditing and troubleshooting.

User prompt: Please help me export your solution above to PDF file

Response: To convert the solution into a PDF file, use any of the quick methods below depending on your setup:

Method 1: Save Directly from Your Browser (Fastest)

1. Select and highlight the text you want to export.
2. Press **Ctrl + P** (Windows) or **Cmd + P** (Mac) to open the print menu.
3. Set the **Destination** or **Printer** drop-down to **Save as PDF**.
4. Under **Pages**, select **Selection** (if you only highlighted a portion) or **All**.
5. Click **Save** and choose your destination folder.

Method 2: Export via Document Editor

1. Copy the text from the response.
2. Paste it into **Google Docs** or **Microsoft Word**.
3. Go to **File > Download** (Google Docs) or **Save As / Export** (Word).
4. Select **PDF Document (.pdf)**.

Method 3: Programmatic Export (Python)

If you are generating solutions programmatically in Python, install fpdf2 (pip install fpdf2) and use the following script:

```
from fpdf import FPDF

# Create PDF instance
pdf = FPDF()
pdf.add_page()
pdf.set_font("Helvetica", size=12)

# Insert text (replace with your content)
text = """Your solution text goes here..."""

pdf.multi_cell(0, 10, text)
pdf.output("solution.pdf")
```
